Module Guide for Software Engineering

Team #1, Sanskrit Ciphers
Omar El Aref
Dylan Garner
Muhammad Umar Khan
Aswin Kuganesan
Yousef Shahin

April 6, 2026

1 Revision History

Date	Version	Notes
January 20	1.0	Added sections 2-5
January 20	1.1	Added sections 6-8
January 20	1.2	Added sections 9-12
April 6	1.3	Added low-fidelity user interface wireframes to support workflow planning and frontend implementation

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CV	Computer Vision
DAG	Directed Acyclic Graph
JSON	JavaScript Object Notation
M	Module
MG	Module Guide
ML	Machine Learning
OCR	Optical Character Recognition
OS	Operating System
R	Requirement
SRS	Software Requirements Specification
UI	User Interface
UC	Unlikely Change

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 3

5 Module Hierarchy **3**

6 Connection Between Requirements and Design **5**

7 Module Decomposition **5**
7.1 Hardware Hiding Modules 5
7.1.1 HW.ImageStorage (M1) 6
7.2 Behaviour-Hiding Module 6
7.2.1 UI.Canvas (M2) 6
7.2.2 UI.Search (M3) 6
7.2.3 UI.Auth (M4) 7
7.2.4 UI.Upload (M5) 7
7.2.5 Svc.API (M6) 7
7.2.6 Svc.Search (M7) 7
7.2.7 Svc.AuthZ (M8) 8
7.2.8 Svc.Session (M9) 8
7.2.9 Svc.ProcOrch (M10) 8
7.2.10 Svc.MatchOrch (M11) 8
7.2.11 Data.FragmentStore (M12) 9
7.2.12 Data.Catalog (M13) 9
7.2.13 Data.User (M14) 9
7.2.14 Data.Project (M15) 9
7.2.15 ML.EdgeMatch (M16) 10
7.2.16 ML.Damage (M17) 10
7.2.17 ML.ScriptClass (M18) 10
7.2.18 ML.OCR (M19) 10
7.2.19 ML.TextSim (M20) 11
7.3 Software Decision Module 11
7.3.1 Util.Error (M21) 11
7.3.2 Util.Logging (M22) 11
7.3.3 Util.Config (M23) 12

7.3.4 Util.TestStub (M24) . . . 12

8 Traceability Matrix **12**

9 Use Hierarchy Between Modules **16**
9.1 Uses Hierarchy Description . . . 17
9.1.1 Layer 5: User Interface . . . 18
9.1.2 Layer 4: API Gateway . . . 18
9.1.3 Layer 3: Service Orchestration . . . 18
9.1.4 Layer 2: Machine Learning . . . 19
9.1.5 Layer 1: Data Storage . . . 19
9.1.6 Layer 0: Foundation . . . 20
9.1.7 Key Architectural Properties . . . 20

10 User Interfaces **21**
10.1 Primary Interface Components . . . 21
10.1.1 Authentication Interface . . . 21
10.1.2 Fragment Upload and Management . . . 21
10.1.3 Interactive Canvas Workspace . . . 22
10.1.4 Search and Filter Interface . . . 22
10.1.5 Matching Suggestions Display . . . 22
10.1.6 Transcription Assistance . . . 22

11 Design of Communication Protocols **23**
11.1 API Architecture . . . 23
11.1.1 Backend API Layer . . . 23
11.2 Core API Endpoints . . . 24
11.2.1 Authentication Endpoints . . . 24
11.2.2 Fragment Management Endpoints . . . 24
11.2.3 Preprocessing Endpoints . . . 24
11.2.4 Matching and Similarity Endpoints . . . 24
11.2.5 OCR and Script Identification Endpoints . . . 24
11.2.6 Session and Workspace Endpoints . . . 25
11.3 Data Exchange Formats . . . 25
11.3.1 Request Schema Example (Fragment Upload) . . . 25
11.3.2 Response Schema Example (Match Suggestions) . . . 25
11.4 Inter-Service Communication . . . 26
11.4.1 Backend to ML/AI Cluster . . . 26
11.4.2 Database Communication . . . 26
11.5 Security and Data Integrity . . . 26

12 Timeline **26**
12.1 Phase 1: Foundation and Infrastructure (Weeks 1-3) . . . 27
12.1.1 Week 1: Project Setup and Foundation Modules . . . 27
12.1.2 Week 2-3: Data Layer Implementation . . . 27
12.2 Phase 2: Service Layer and API (Weeks 4-6) . . . 27
12.2.1 Week 4: Authentication and Session Management . . . 27
12.2.2 Week 5-6: Core Service Modules . . . 28
12.3 Phase 3: Machine Learning Integration (Weeks 7-10) . . . 28
12.3.1 Week 7-8: ML Infrastructure and Basic Models . . . 28
12.3.2 Week 9-10: Advanced ML Models . . . 28
12.4 Phase 4: User Interface Development (Weeks 8-11) . . . 29
12.4.1 Week 8-9: Core UI Components . . . 29
12.4.2 Week 10-11: Interactive Canvas . . . 29
12.5 Phase 5: Testing and Refinement (Weeks 12-14) . . . 30
12.5.1 Week 12: Integration Testing . . . 30
12.5.2 Week 13: Performance Optimization and ML Tuning . . . 30
12.5.3 Week 14: User Acceptance Testing and Documentation . . . 30
12.6 Phase 6: Deployment and Monitoring (Week 15) . . . 30

List of Tables

1 Module Hierarchy . . . 4
2 Trace Between Requirements and Modules . . . 13
3 Trace Between Anticipated Changes and Modules . . . 14

List of Figures

1 Use hierarchy among modules (simplified view showing key dependencies) . . . 16
2 User Interface Flow Diagram showing the complete interaction workflow from login to fragment analysis . . . 23

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules layed out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS) (**?**), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: The specific hardware on which the software is running.

AC2: The format of the image input data (file formats, metadata structure).

AC3: The authentication and authorization mechanism for different user types.

AC4: The method for uploading and storing fragment images.

AC5: The algorithm used for searching and filtering fragments.

AC6: The machine learning model used for edge matching detection.

AC7: The algorithm for fragment segmentation and feature extraction.

AC8: The method for script classification and similarity matching.

AC9: The algorithm for text similarity comparison.

AC10: The algorithm for detecting and counting text lines in fragments.

AC11: The algorithm for detecting circular marks and patterns.

AC12: The algorithm for classifying edge types (torn, cut, original, burned).

AC13: The user interface framework for the interactive canvas.

AC14: The database schema and storage mechanism for fragment data.

AC15: The structure of catalog metadata.

AC16: The orchestration logic for combining multiple ML model results.

AC17: The session management and state persistence mechanism.

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Input/Output devices (Input: Image files via web upload, Output: Screen display and file export). This is unlikely to change because the system is designed specifically for standard web-based input/output devices, and supporting alternative hardware would require a fundamental redesign.

UC2: The system will be deployed as a web-based application accessible through standard browsers. This is unlikely to change because the system is intended to be accessed through browsers, and changing the deployment model would significantly alter the architecture.

UC3: The system will process 2D images of manuscript fragments. This is unlikely to change because the system is specifically designed for 2D manuscript fragment analysis, and extending to other formats would require major redesign.

UC4: The system will support collaborative work through shared projects and annotations. This is unlikely to change because collaboration is a core requirement of the system and removing it would contradict the primary use case.

UC5: Machine learning will be a core component for fragment matching and analysis. This is unlikely to change because the system fundamentally relies on machine learning techniques for reconstruction and analysis, and removing or replacing this would require a complete redesign of the system's core functionality.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets, shown in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: HW.ImageStorage Module

M2: UI.Canvas Module

M3: UI.Search Module

M4: UI.Auth Module

M5: UI.Upload Module

M6: UI.SaveSession Module

M7: UI.LoadSession Module

M8: Svc.API Module

M9: Svc.Search Module

M10: Svc.AuthZ Module

M11: Svc.Session Module

M12: Svc.ProcOrch Module

M13: Svc.MatchOrch Module

M14: Data.FragmentStore Module

M15: Data.Catalog Module

M16: Data.User Module

M17: Data.Project Module

M18: ML.EdgeMatch Module

M19: ML.ScriptClass Module

M20: ML.TextSim Module

M21: ML.Segmentation Module

M22: ML.CircleRecognition Module

M23: ML.LineCount Module

M24: ML.EdgeClassification Module

M25: Error Handling Module

M26: Logging Module

M27: Configuration Management Module

M28: Test Stubs Module

Table 1: Module Hierarchy

Level 1	**Level 2**
Hardware-Hiding	HW.ImageStorage
Behaviour-Hiding	*User Interface* UI.Canvas, UI.Search, UI.Auth, UI.Upload, UI.SaveSession, UI.LoadSession
	Service Layer Svc.API, Svc.Search, Svc.AuthZ, Svc.Session, Svc.ProcOrch, Svc.MatchOrch
	Data Management Data.FragmentStore, Data.Catalog, Data.User, Data.Project
	Machine Learning ML.EdgeMatch, ML.ScriptClass, ML.TextSim, ML.Segmentation, ML.CircleRecognition, ML.LineCount, ML.EdgeClassification
Software Decision	Utility and Support Modules: Error Handling, Logging, Configuration Management, Test Stubs

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

FR1.1 (User Authentication) is implemented through the M4 and M8 modules which manage credential verification and secure access control. Similarly, FR1.2 (Fragment Management) and FR1.3 (Display Fragments) are supported by the M2, M5, and M12 modules that enable fragment visualization and organization. Session management is enhanced by M**??** and M**??** modules for workspace persistence. The ML-oriented requirements FR3.1-FR3.3 are implemented by specialized modules such as M16, M18, M**??**, and M**??**, coordinated by M11.

Non-functional requirements are distributed across multiple layers of the system. NFR1.1 (Usability) and NFR1.2 (Performance) are primarily addressed by front-end and orchestration modules to ensure responsiveness and ease of interaction. NFR2.1 (Scalability) and NFR2.2 (Reliability) are achieved through the use of modular orchestration, structured data

storage, and robust error handling in modules such as M10, M13, and M21. Finally, NFR3.1-NFR3.3 (Accuracy, Efficiency, and Maintainability) govern the design of the AI/ML cluster, ensuring that the models (M16, M??, M20, M??, M??, M??) can be retrained, tuned, and maintained without disrupting other components.

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by Parnas et al. (1984). The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *Software Engineering* means the module will be implemented by the Software Engineering software. The *Type of Module* field is used to explain why each module is shaped the way it is: *Abstract Object* is used for stateful services, UI components, and ML components with hidden behaviour; *Abstract Data Type* is used where operations must protect internal data representations; *Record* is used for modules that primarily model structured stored entities; and *Library* is used for shared, largely stateless helper functionality.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware Hiding Modules

Secrets: The data structure and algorithm used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. This module provides the interface between the hardware and the software. So, the system can use it to display outputs or to accept inputs.

Implemented By: OS

7.1.1 HW.ImageStorage (M1)

Secrets: Data structure and algorithms used to implement the image storage

Services: This module provides platform-specific file operations such as image storage and retrieval, and it serves as virtual hardware for the system.

Implemented By: Cloud or Local Storage Interface

Type of Module: Abstract Object

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 UI.Canvas (M2)

Secrets: The implementation of the interactive canvas for fragment manipulation.

Services: Provides an interactive interface to arrange multiple fragments and displays match suggestions

Implemented By: Front-End Web Client (React/Next.js component)

Type of Module: Abstract Object

7.2.2 UI.Search (M3)

Secrets: The user interface design for search and filtering functionality.

Services: Provides a search interface for fragments, and supports search by script type, line count, and fragment size.

Implemented By: Front-End Web Client (React/Next.js component)

Type of Module: Abstract Object

7.2.3 UI.Auth (M4)

Secrets: The user interface for authentication and user management.

Services: Provides login/logout interface, user registration, role-based access control UI, and session management display.

Implemented By: Front-End Authentication Component

Type of Module: Abstract Object

7.2.4 UI.Upload (M5)

Secrets: The user interface for uploading fragment images and metadata.

Services: Provides a file upload interface to select and submit images to the backend for processing after validation.

Implemented By: Front-End Web Client (React/Next.js component)

Type of Module: Abstract Object

7.2.5 UI.SaveSession (M??)

Secrets: The user interface for saving workspace state and session data.

Services: Provides a dialog interface to name and save the current workspace state, including fragment arrangements and annotations, for later retrieval.

Implemented By: Front-End Web Client (React/Next.js component)

Type of Module: Abstract Object

7.2.6 UI.LoadSession (M??)

Secrets: The user interface for browsing and loading saved sessions.

Services: Provides a dialog interface to browse, select, and load previously saved workspace sessions, displaying session metadata and enabling restoration of reconstruction work.

Implemented By: Front-End Web Client (React/Next.js component)

Type of Module: Abstract Object

7.2.7 Svc.API (M6)

Secrets: The RESTful API endpoint definitions and routing logic.

Services: Sends incoming HTTP requests from the UI to services and handles request validation

Implemented By: Backend API Gateway (FastAPI, Flask, or Node.js)

Type of Module: Abstract Object

7.2.8 Svc.Search (M7)

Secrets: The search algorithm and query optimization logic.

Services: This module performs efficient fragment searches over metadata and fragments, and it can export the results as a PDF or CSV file

Implemented By: Backend Search Service

Type of Module: Abstract Object

7.2.9 Svc.AuthZ (M8)

Secrets: The authentication and authorization implementation.

Services: This module validates user credentials and tokens, and it enforces access control for protected operations

Implemented By: Authentication & Authorization Microservice

Type of Module: Abstract Object

7.2.10 Svc.Session (M9)

Secrets: The session state management and persistence mechanism.

Services: Manages user sessions and provides storage and retrieval of the saved user workspace

Implemented By: Backend Session Manager

Type of Module: Abstract Object

7.2.11 Svc.ProcOrch (M10)

Secrets: The orchestration logic for image processing pipelines.

Services: Coordinates preprocessing workflows for images uploaded by user, and manages job queues for ML tasks

Implemented By: Workflow Orchestrator

Type of Module: Abstract Object

7.2.12 Svc.MatchOrch (M11)

Secrets: The algorithm for combining results from multiple ML models.

Services: Aggregates and sorts results from all ML models and provides confidence scores

Implemented By: Backend Orchestrator Service

Type of Module: Abstract Object

7.2.13 Data.FragmentStore (M12)

Secrets: The database schema and queries for fragment storage.

Services: Stores and retrieves fragment entities such as fragment IDs, associated image IDs, script labels, line counts, and status flags

Implemented By: Data Access Layer

Type of Module: Abstract Data Type

7.2.14 Data.Catalog (M13)

Secrets: The catalog metadata structure and taxonomy.

Services: Stores mappings between internal fragments and external catalogue identifiers

Implemented By: Metadata Database

Type of Module: Abstract Data Type

7.2.15 Data.User (M14)

Secrets: The user data model and storage.

Services: Stores all user information such as accounts, credentials, roles and profile information for authentication and access control purposes

Implemented By: User Account Database Table

Type of Module: Record

7.2.16 Data.Project (M15)

Secrets: The project data structure for collaborative work.

Services: Manages project workspaces and collections and stores project-specific fragment groupings

Implemented By: Project Database Schema

Type of Module: Record

7.2.17 ML.EdgeMatch (M16)

Secrets: Model details including weights and hyperparameters, segmentation mask and edge matching algorithms.

Services: Provides edge match similarity probability and computes edge features for images.

Implemented By: AI/ML Cluster (Python-based ML Service)

Type of Module: Abstract Object

7.2.18 ML.ScriptClass (M18)

Secrets: Model architecture and hyperparameters, and script label and metadata.

Services: Determines the type of script based on text in the script and provides the script label with a confidence score.

Implemented By: AI/ML Cluster (Text Classification Service)

Type of Module: Abstract Object

7.2.19 ML.TextSim (M20)

Secrets: Vectorization and normalization methods, and algorithm used for checking similarity

Services: Converts text to embedding vector and compares similarity between text segments using transcribed text or manual annotations

Implemented By: AI/ML Cluster (Text Similarity Service)

Type of Module: Abstract Object

7.2.20 ML.Segmentation (M??)

Secrets: Deep learning model weights and configuration for image segmentation.

Services: Performs image segmentation to identify and isolate individual fragment regions from manuscript images, detecting fragment boundaries and separating multiple fragments from a single scanned image.

Implemented By: AI/ML Cluster (Python-based ML Service)

Type of Module: Abstract Object

7.2.21 ML.CircleRecognition (M??)

Secrets: Detection parameters and thresholds for circle recognition algorithms.

Services: Detects circular marks, holes, or decorative elements in manuscript fragments. These features serve as additional matching criteria when aligning fragments.

Implemented By: AI/ML Cluster (Python-based ML Service)

Type of Module: Abstract Object

7.2.22 ML.LineCount (M??)

Secrets: Line detection model parameters and spacing thresholds.

Services: Analyzes manuscript fragments to count the number of text lines present. Line count serves as a useful filtering and matching criterion during fragment search and reconstruction.

Implemented By: AI/ML Cluster (Python-based ML Service)

Type of Module: Abstract Object

7.2.23 ML.EdgeClassification (M??)

Secrets: Edge classification model weights and feature extraction methods.

Services: Classifies fragment edges into categories such as torn, cut, original (manuscript boundary), or burned. Edge type classification improves matching accuracy by ensuring compatible edge pairs are compared.

Implemented By: AI/ML Cluster (Python-based ML Service)

Type of Module: Abstract Object

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Util.Error (M21)

Secrets: The error handling and exception management plan

Services: Defines standard error types and provides centralized error handling across all modules

Implemented By: Shared Backend Utility Package

Type of Module: Library

7.3.2 Util.Logging (M22)

Secrets: The logging implementation and structure of the log format

Services: This module records system events and user actions, and it provides structured logging for information, warnings and errors.

Implemented By: Shared Logging Framework

Type of Module: Library

7.3.3 Util.Config (M23)

Secrets: The configuration format and loading mechanism

Services: Loads configuration from files and provides read-only access to configuration values by providing the value or boolean based on key given.

Implemented By: Configuration Manager

Type of Module: Library

7.3.4 Util.TestStub (M24)

Secrets: The mock implementations and mock model outputs

Services: This module provides stub implementations of services and ML components, and it simulates ML model responses

Implemented By: Test Utilities

Type of Module: Library

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	Modules
FR1.1	M4, M8, M9, M14
FR1.2	M2, M5, M6, M10, M12, M13, M??
FR1.3	M2, M3, M6, M??
FR1.4	M2, M11, M16, M18, M20, M12, M??, M??
FR1.5	M2, M4, M9, M15, M??, M??
NFR1.1	M2, M3, M4, M5, M??, M??
NFR1.2	M1, M2, M5, M6
FR2.1	M6, M8, M9, M11
FR2.2	M8, M9, M14
FR2.3	M12, M13, M14, M15
NFR2.1	M6, M10, M11, M1
NFR2.2	M12, M13, M14, M15, M21, M22, M23
FR3.1	M16, M11, M??
FR3.2	M18, M11
FR3.3	M??, M11, M??, M??
NFR3.1	M16, M18, M20, M??, M??, M??, M??
NFR3.2	M16, M18, M20, M??, M??, M??, M??, M10, M11
NFR3.3	M16, M18, M20, M??, M??, M??, M??, M22, M23, M24

Table 2: Trace Between Requirements and Modules

AC	**Modules**
AC1	M1, M23
AC2	M5, M10, M1
AC3	M4, M8, M14, M9, M6
AC4	M5, M10, M12
AC5	M3, M7, M13
AC6	M16, M11, M??
AC7	M??, M11
AC8	M18, M11
AC10	M20, M11
AC11	M2, M3, M5, M??, M??
AC12	M12, M13, M15
AC13	M13, M7
AC14	M11, M16, M18, M20, M??, M??, M??, M??
AC15	M9, M14, M15, M8, M??, M??
AC??	M??, M11, M7
AC??	M??, M11
AC??	M??, M11, M16

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

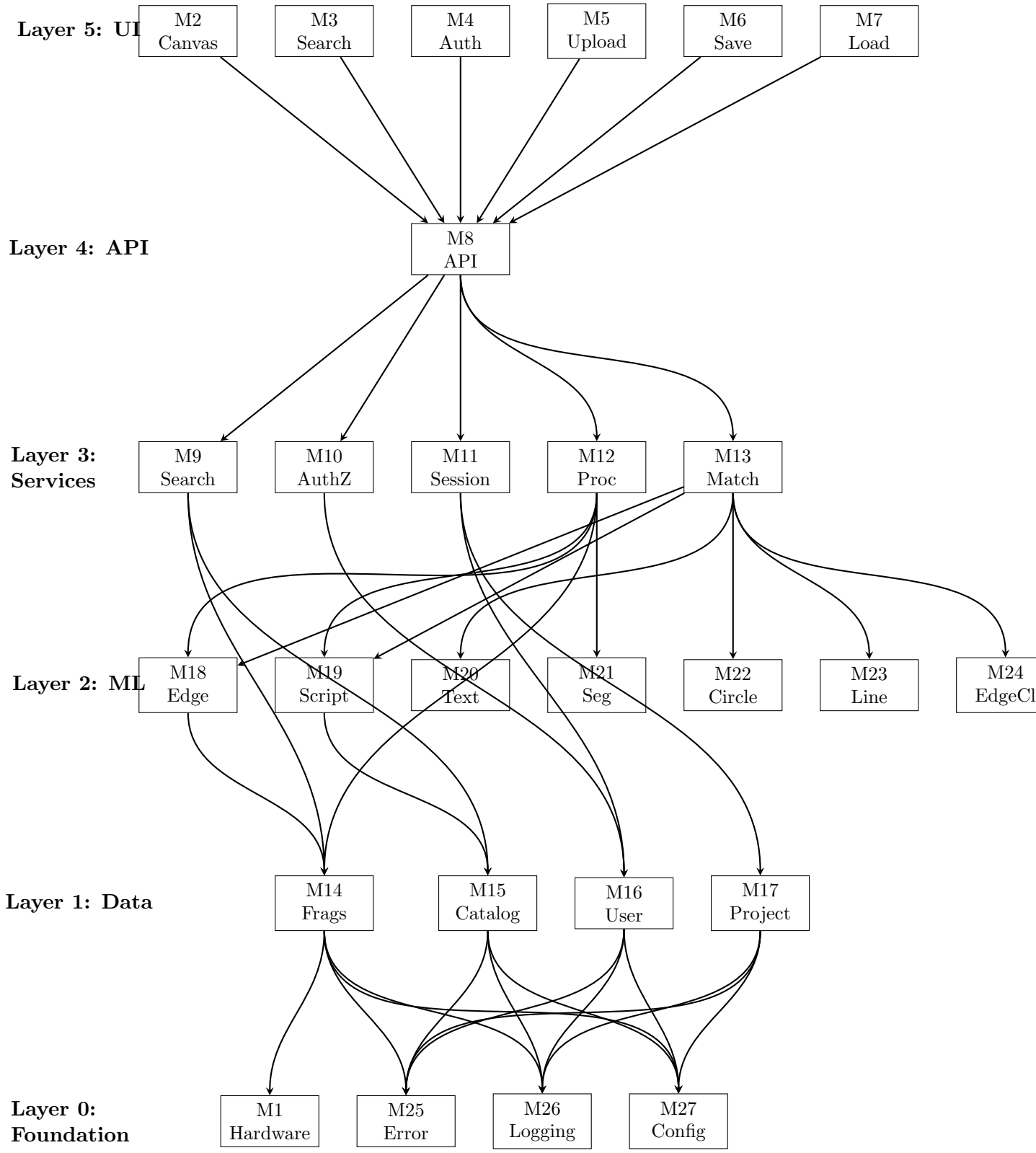

Figure 1: Use hierarchy among modules (simplified view showing key dependencies)

9.1 Uses Hierarchy Description

The uses hierarchy follows a strict layered architecture where modules at each level depend only on services provided by lower levels. The diagram shows direct dependencies between modules, with arrows indicating the uses relationship.

9.1.1 Layer 5: User Interface

All UI modules communicate exclusively through the API layer:

- **Canvas (M2), Search (M3), Auth (M4), Upload (M5), SaveSession (M6), LoadSession (M7)** all directly use **API (M8)**
- UI modules have no direct access to service, data, or ML layers, ensuring clean separation between presentation and business logic
- **SaveSession (M6)** and **LoadSession (M7)** provide dedicated interfaces for workspace persistence and retrieval

9.1.2 Layer 4: API Gateway

API (M8) serves as the central gateway and coordinates all service layer modules:

- Directly uses all five service modules: **SearchSvc (M9)**, **AuthZ (M10)**, **Session (M11)**, **ProcOrch (M12)**, and **MatchOrch (M13)**
- Provides unified RESTful interface to all backend services for the UI layer
- Handles request routing, validation, response formatting, and rate limiting
- Acts as the single entry point for all business logic operations

9.1.3 Layer 3: Service Orchestration

Service modules implement business logic and coordinate lower-level modules:

Data Access Services:

- **SearchSvc (M9)** uses **FragmentStore (M14)** and **Catalog (M15)** to perform search and filtering operations across fragments
- **AuthZ (M10)** uses **User (M16)** for credential verification and role-based authorization
- **Session (M11)** uses **User (M16)** and **Project (M17)** to manage user sessions and collaborative workspace state

Orchestration Services:

- **ProcOrch (M12)** orchestrates image processing pipelines by using:
  - **FragmentStore (M14)** for accessing fragment images
  - **EdgeMatch (M18)** for edge detection during preprocessing
  - **ScriptClass (M19)** for initial script classification
  - **Segmentation (M21)** for isolating individual fragments from images
- **MatchOrch (M13)** coordinates fragment matching by using:
  - **EdgeMatch (M18)** for edge profile comparison
  - **ScriptClass (M19)** for script similarity comparison
  - **TextSim (M20)** for comparing extracted text between fragments
  - **CircleRecognition (M22)** for matching circular marks and holes
  - **LineCount (M23)** for filtering by text line count
  - **EdgeClassification (M24)** for classifying edge types (torn, cut, original, burned)

Note that ProcOrch focuses on preprocessing and feature extraction, while MatchOrch focuses on comparing fragments and ranking matches.

9.1.4 Layer 2: Machine Learning

ML modules provide specialized analysis capabilities:

- **EdgeMatch (M18)** uses **FragmentStore (M14)** to access fragment images for edge detection and comparison
- **ScriptClass (M19)** uses both **Catalog (M15)** for script taxonomy metadata and **FragmentStore (M14)** for fragment images
- **Segmentation (M21)** performs image segmentation to isolate individual fragments from scanned images
- **CircleRecognition (M22)** detects circular marks, holes, or decorative elements for matching
- **LineCount (M23)** counts text lines for filtering and matching criteria
- **EdgeClassification (M24)** classifies fragment edges (torn, cut, original, burned) to improve matching accuracy
- Other ML modules (**TextSim (M20)**) also use **FragmentStore (M14)** for data access (connections shown representatively to reduce visual complexity)
- ML modules are always coordinated by service orchestrators (ProcOrch or MatchOrch), never invoked directly by the API

9.1.5 Layer 1: Data Storage

Data modules encapsulate all persistence operations and provide CRUD interfaces:

- **FragmentStore (M14)** uses:
  - **Hardware Hiding (M1)** for file system and storage operations
  - **Error Handling (M25)** for exception management
  - **Logging (M26)** for audit trails of fragment operations
  - **Config (M27)** for storage configuration parameters
- **Catalog (M15)** uses **Error Handling (M25)**, **Logging (M26)**, and **Config (M27)** for managing controlled vocabularies and metadata schemas
- **User (M16)** uses **Error Handling (M25)**, **Logging (M26)**, and **Config (M27)** for user profile management and authentication data
- **Project (M17)** uses **Error Handling (M25)**, **Logging (M26)**, and **Config (M27)** for collaborative workspace data

All data modules depend heavily on foundation modules to ensure reliable, configurable, and auditable data operations.

9.1.6 Layer 0: Foundation

Foundation modules provide cross-cutting system services with no dependencies on other system modules:

- **Hardware Hiding (M1)** abstracts operating system and file system operations, used by FragmentStore for persistent storage
- **Error Handling (M25)** provides exception management and error reporting, used by all data modules and propagated throughout upper layers
- **Logging (M26)** records system events, user actions, and audit trails, used extensively by data and service layers
- **Config (M27)** manages application configuration from files and environment variables, used by all data modules and some services

9.1.7 Key Architectural Properties

The uses hierarchy exhibits several important design properties:

1. **Strict Layering:** All dependencies flow downward through the layer hierarchy. Modules only use modules in lower layers, never peers or higher layers, ensuring clear separation of concerns.

2. **API Gateway Pattern:** The API module (M8) serves as the single entry point for all UI modules, providing a unified interface to backend services and preventing direct UI-to-service coupling.

3. **Orchestration Pattern:** Complex workflows involving multiple ML modules are coordinated by dedicated orchestrator services (ProcOrch and MatchOrch), which hide the complexity of multi-model coordination from the API layer.

4. **Data Encapsulation:** All persistent data access flows through dedicated data modules (M14-M17), preventing direct database access from service or ML layers and enabling consistent data validation and access control.

5. **Foundation Ubiquity:** Utility modules (Error, Logging, Config) are used throughout the system, particularly by the data layer. The diagram shows representative uses of these modules by all data modules, though they are actually used more broadly throughout the system.

6. **DAG Structure:** The dependency graph forms a directed acyclic graph with no circular dependencies, which enables:

   - Independent development and testing of each layer
   - Incremental system integration from bottom to top
   - Module substitution without affecting higher layers
   - Clear understanding of system dependencies and impact analysis

7. **Testable Subsets:** Each layer represents a testable subset of the system. Layer 0 can be tested independently, Layer 1 requires only Layer 0, and so on, supporting systematic integration testing.

8. **Separation of Concerns:** The architecture clearly separates presentation (Layer 5), API interface (Layer 4), business logic (Layer 3), specialized algorithms (Layer 2), persistence (Layer 1), and system utilities (Layer 0).

This hierarchical organization supports maintainability, testability, and evolution of the system over time.

10 User Interfaces

The Sanskrit Manuscript Reconstruction Platform provides a web-based interface designed for Buddhist Studies scholars to interact with manuscript fragments. The interface is built using React with TypeScript and styled using CSS to ensure consistency and responsiveness.

10.1 Primary Interface Components

10.1.1 Authentication Interface

- **Login Screen:** Secure credential entry with username and password fields
- **Session Management:** Token-based authentication with automatic session persistence
- **Error Handling:** Clear error messages for invalid credentials ("Invalid username or password")

10.1.2 Fragment Upload and Management

- **Upload Interface:** Drag-and-drop or file browser support for image uploads (JPG, PNG, TIFF formats, max 50 MB)
- **Preprocessing Visualization:** Real-time display of orientation correction with confidence scores
- **Fragment Browser:** Thumbnail grid view of uploaded fragments with metadata overlay

10.1.3 Interactive Canvas Workspace

- **Drag-and-Drop Canvas:** Visual workspace for arranging and comparing fragments
- **Fragment Manipulation:** Rotation, zoom, and positioning controls with snap-to-edge functionality
- **Session Persistence:** Save and restore workspace arrangements
- **Response Time Target:** UI drag/drop feedback latency under 200 ms

10.1.4 Search and Filter Interface

- **Search Bar:** Fragment ID search with autocomplete
- **Metadata Filters:** Multi-criteria filtering (script type, line count, fragment size)
- **Results Display:** Grid or list view with sortable columns
- **Export Options:** Download filtered results as CSV or PDF

10.1.5 Matching Suggestions Display

- **Ranked Match List:** Top-ranked fragment matches with similarity scores and confidence levels
- **Visual Comparison:** Side-by-side or overlay view for comparing fragments
- **Trust Signals:** Confidence scores, edge pattern overlays, and match explanations

10.1.6 Transcription Assistance

- **Script Classification:** Display of identified script type (e.g., Gupta, Tibetan, Chinese) with confidence $\geq 70\%$
- **Manual Correction:** Editable text fields for scholarly review and correction

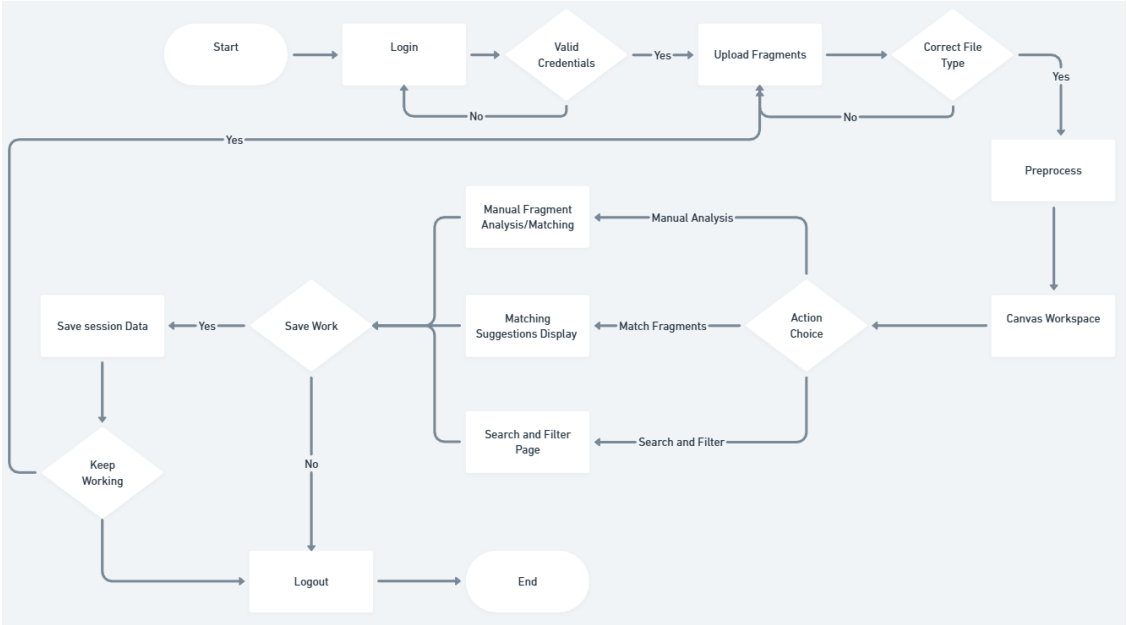

Figure 2: User Interface Flow Diagram showing the complete interaction workflow from login to fragment analysis

11 Design of Communication Protocols

The Sanskrit Manuscript Reconstruction Platform uses a client-server architecture in which the user interface communicates with backend services through a versioned service boundary. This section describes communication at the module-interface level, focusing on responsibilities and information flow rather than implementation syntax.

11.1 API Architecture

11.1.1 Backend API Layer

The backend API gateway provides a stable interface between UI modules and service modules. Its role is to route requests to the correct service module, enforce cross-cutting policies, and return standardized responses.

API Design Principles:

- **Versioned Interface:** Service contracts are versioned to allow interface evolution without breaking existing clients.
- **Uniform Operations:** Interface operations are defined in terms of create, read, update, delete, and query actions over system resources.

- **Standardized Responses:** All operations return a consistent response structure with either requested data or a categorized error.
- **Input and Output Validation:** All module boundaries validate incoming arguments and outgoing results against agreed contracts.
- **Separation of Concerns:** Transport concerns are isolated in the API gateway while domain logic remains in service modules.

11.2 Core API Capabilities

At the architectural level, API capabilities are grouped by service responsibility rather than by concrete route definitions:

- **Authentication and Authorization Operations:** establish and validate user identity, and enforce access permissions.
- **Fragment Lifecycle Operations:** ingest, retrieve, query, and remove fragment records and associated assets.
- **Preprocessing Operations:** initiate and monitor image normalization workflows.
- **Matching and Analysis Operations:** request candidate matches and retrieve ranked similarity outcomes.
- **Session and Workspace Operations:** create, update, restore, list, and remove saved workspace states.

11.3 Data Exchange Formats

Communication between modules uses typed request and response contracts with clear semantic fields.

11.3.1 Request Contract Categories

- **Identity and Access Inputs:** credentials, session tokens, and authorization context.
- **Fragment Inputs:** image payload references and descriptive metadata required for storage and indexing.
- **Query Inputs:** search filters, sorting choices, and pagination parameters.
- **Workflow Inputs:** identifiers used to start or inspect preprocessing and matching jobs.

11.3.2 Response Contract Categories

- **Operation Result:** requested resource data or confirmation of completed action.
- **Analysis Result:** ranked candidate lists with confidence or score summaries.
- **Workflow Status:** lifecycle state, progress indicators, and completion metadata.
- **Error Result:** normalized error category and diagnostic message suitable for UI handling.

11.4 Inter-Service Communication

11.4.1 Backend to ML/AI Cluster

The backend orchestration services communicate with ML components through internal service contracts (synchronous calls for immediate results and asynchronous jobs for long-running analysis):

- **Text and Script Analysis:** model inference requests and prediction responses.
- **Similarity and Matching:** feature and score exchange for candidate ranking.
- **Long-Running Jobs:** queued execution with explicit status retrieval.

11.4.2 Database Communication

- **Data Access Abstraction:** persistence is encapsulated behind data modules to isolate storage technology decisions.
- **Query Performance Support:** indexing and query planning are applied to satisfy search and retrieval constraints.
- **Concurrency Support:** data access is designed to support concurrent user workflows and collaborative use.

11.5 Security and Data Integrity

- **Authentication:** Session and token-based identity verification.
- **Authorization:** Role-based access control for sensitive operations
- **Input Validation:** Server-side validation of all API inputs
- **Data Access Safety:** Protected persistence operations through validated and parameterized data access patterns.
- **API Rate Limiting:** Throttling to prevent abuse
- **HTTPS:** Encrypted communication in production deployment

12 Timeline

The development of the Sanskrit Manuscript Reconstruction Platform follows a phased approach, with module implementation organized by architectural layer. The timeline ensures that foundational components are established before dependent modules are developed, following the uses hierarchy described in Section 9.

12.1 Phase 1: Foundation and Infrastructure (Weeks 1-3)

12.1.1 Week 1: Project Setup and Foundation Modules

- **Setup Development Environment**
  - Configure MongoDB, Node.js/Express backend, and React frontend repositories
  - Set up version control, CI/CD pipelines, and development/staging environments
  - *Responsible: Full Team*
- **Implement Foundation Modules (Layer 0)**
  - M21: Error Handling Module - Define error types, implement centralized exception handling
  - M22: Logging Module - Configure structured logging framework (Winston/Bunyan)
  - M23: Configuration Management - Implement environment-based configuration loading
  - *Responsible: Yousef Shahin, Aswin Kuganesan*

12.1.2 Week 2-3: Data Layer Implementation

- **Hardware-Hiding Module**
  - M1: Implement file storage abstraction (local/cloud storage interface)
  - *Responsible: Omar El Aref*
- **Data Modules (Layer 1)**
  - M14: User database schema, authentication data models
  - M12: Fragment storage schema, image metadata, CRUD operations
  - M13: Catalog metadata structure, controlled vocabularies
  - M15: Project workspace schema, collaborative annotations
  - *Responsible: Umar Khan, Dylan Garner*

12.2 Phase 2: Service Layer and API (Weeks 4-6)

12.2.1 Week 4: Authentication and Session Management

- **Authentication Services (Layer 3)**
  - M8: JWT-based authentication, role-based access control
  - M9: Session state management, workspace persistence
  - *Responsible: Yousef Shahin, Aswin Kuganesan*
- **API Gateway (Layer 4)**
  - M6: Express.js RESTful API setup, authentication endpoints
  - *Responsible: Omar El Aref*

12.2.2 Week 5-6: Core Service Modules

- **Service Orchestration (Layer 3)**
  - M7: Fragment search and filtering logic, query optimization
  - M10: Image preprocessing pipeline orchestration, job queue management
  - M11: ML model result aggregation, confidence scoring (initial implementation without ML models)
  - *Responsible: Umar Khan, Dylan Garner*
- **API Endpoint Completion**
  - Complete all RESTful endpoints for fragment management, search, sessions, and preprocessing
  - *Responsible: Omar El Aref, Yousef Shahin*

12.3 Phase 3: Machine Learning Integration (Weeks 7-10)

12.3.1 Week 7-8: ML Infrastructure and Basic Models

- **ML Service Setup**
  - Python ML service infrastructure, REST API for ML models
  - Model serving framework (Flask/FastAPI for Python services)
  - *Responsible: Aswin Kuganesan, Umar Khan*
- **Initial ML Modules (Layer 2)**
  - M16: Edge detection and matching algorithm implementation
  - M18: Script classification model training and deployment
  - *Responsible: Dylan Garner, Yousef Shahin*

12.3.2 Week 9-10: Advanced ML Models

- **ML Module Completion (Layer 2)**
  - M17: Damage pattern analysis and similarity scoring
  - M19: OCR model integration (using existing libraries like Tesseract/EasyOCR, fine-tuned for ancient scripts)
  - M20: Text similarity comparison using embeddings
  - *Responsible: Omar El Aref, Aswin Kuganesan*
- **Orchestrator Integration**
  - Connect ML modules to M10 and M11
  - Implement weighted scoring and confidence calculation
  - *Responsible: Umar Khan, Dylan Garner*

12.4 Phase 4: User Interface Development (Weeks 8-11)

Note: UI development runs partially in parallel with ML integration

12.4.1 Week 8-9: Core UI Components

- **Authentication UI (Layer 5)**
  - M4: Login/logout interface, user registration, session display
  - *Responsible: Yousef Shahin*
- **Fragment Management UI**
  - M5: File upload interface with drag-and-drop, metadata forms
  - M3: Search bar, filtering interface, results display
  - *Responsible: Omar El Aref, Aswin Kuganesan*

12.4.2 Week 10-11: Interactive Canvas

- **Canvas Workspace (Layer 5)**
  - M2: Drag-and-drop canvas for fragment arrangement
  - Real-time match visualization, collaborative workspace features
  - Integration with M11 for displaying suggestions
  - *Responsible: Umar Khan, Dylan Garner*

12.5 Phase 5: Testing and Refinement (Weeks 12-14)

12.5.1 Week 12: Integration Testing

- **Test Infrastructure**
  - Mock implementations for all services and ML components
  - *Responsible: Full Team*
- **Layer-by-Layer Integration Testing**
  - Test data layer with foundation modules
  - Test service layer with data layer
  - Test ML integration with orchestrators
  - Test UI with complete backend
  - *Responsible: Full Team*

12.5.2 Week 13: Performance Optimization and ML Tuning

- Database query optimization, API response time tuning
- ML model fine-tuning based on test fragment dataset
- Concurrent user testing (target: 15+ users)
- *Responsible: Yousef Shahin, Aswin Kuganesan, Omar El Aref*

12.5.3 Week 14: User Acceptance Testing and Documentation

- User testing with domain experts (Buddhist Studies scholars)
- Bug fixes and usability improvements
- Complete API documentation
- User manual and deployment documentation
- *Responsible: Full Team*

12.6 Phase 6: Deployment and Monitoring (Week 15)

- Production environment setup and deployment
- Monitoring and logging infrastructure configuration
- Performance monitoring and error tracking setup

- Handoff and training for stakeholders
- *Responsible: Full Team*

Note: For detailed task assignments, sprint planning, and issue tracking, refer to the project's GitHub repository at: `https://github.com/DylanG5/sanskrit-cipher`

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78: Proceedings of the 3rd international conference on Software engineering*, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.